

Supplementary Materials ([Automated Financial Risk Assessment Notebook](#))

Implementation details

This section outlines the implementation of the risk severity classification system, including preprocessing steps, model configurations and training environment setup.

The implementation was carried out in a Google Colab environment using Python 3 and a CPU or GPU depending on the model being used. The implementation was carried using key libraries such as pandas and NumPy for data manipulation, nltk for preprocessing and lemmatization, and scikit-learn for model training and evaluation. Transformer-based models were implemented via the transformers library, with torch. Visualizations were generated using matplotlib, seaborn, and WordCloud. For keyword extraction and semantic analysis, KeyBERT and YAKE. The datasets library from Hugging Face was used to load the financial risk dataset.

Initial exploratory analysis revealed clearer term patterns in unigrams and bigrams, prompting the use of a (1, 2) n-gram range in vectorizers. Trigrams showed high overlap and sparsity across severity levels and were therefore excluded.

Table 1: Top Bigrams by severity

	HIGH	MEDIUM	LOW	NONE
1	results operations	results operations	foreign currency	financial reporting
2	financial condition	laws regulations	ended december	financial statements
3	laws regulations	financial condition	common stock	internal control
4	condition results	products services	cash cash	control financial
5	material adverse	fair value	cash equivalents	fair value

Table 2: Top Trigrams by severity

	HIGH	MEDIUM	LOW	NONE
1	financial condition results	financial condition results	cash cash equivalents	internal control financial
2	condition results operations	condition results operations	year ended december	control financial reporting
3	material adverse effect	year ended december	deferred tax assets	company internal control
4	business financial condition	business financial condition	position results operations	current report form
5	could material adverse	material adverse effect	foreign currency exchange	year ended december

As observed in Table 1 above, bigrams capture most domain-relevant phrases. Trigrams would also lead to more noise as our dataset is relatively small and imbalanced. We can see from Table 2, that trigrams are extensions of common bigrams which would not add more meaning.

Another important aspect of the implementation of our models is the text augmentation pipeline. We attempted to use the 'nlpaug' library, but this led to high runtime on our models which we could have investigated further. But, due to the runtime limitations, we opted to implement a custom synonym replacement function using WordNet through the NLTK library.

Our pipeline targets the two minority classes (LOW and NONE) to address class imbalance. For each of these classes, a subset of samples was randomly selected, and approximately 20% of words in each sentence were replaced with randomly sampled synonyms. To ensure meaningful replacements, only alphabetic words longer than three characters were considered, and replacements were limited to words with available synonyms in WordNet. The augmented samples were then combined with the original dataset to form a more balanced training set.

For the implementation of the base models, we explored TF-IDF, BoW and GloVe as text vectorizers. vocabulary size limited to the top 5,000 features (max_features=5000) and a minimum document frequency threshold of 5 (min_df=5) to exclude rare terms. These sparse representations were well-suited for traditional classifiers like Logistic Regression and SVM. In contrast, GloVe embeddings were used to generate dense, 300-dimensional word vectors. Performance varied slightly across vectorizers, with TF-IDF generally outperforming BoW, while GloVe showed weaker results likely due to its static, context-agnostic nature.

The different text vectorizers were applied on machine learning models (Logistic regression and SVM). We identified that SVM with class weights set to 'balanced' and with an RBF kernel led to a more balanced performance across the classes. LinearSVC showed skewed performance towards the majority classes, but runtime was significantly lower than the RBF kernel. TF-IDF with SVM outperformed all other model combinations; we attempted to further optimise the model through Grid Search where explores different regularization strength (C), different kernels, class weights set to 'None' and 'balanced' and different gamma values. We set the grid search to optimise the model on F1-score due to imbalanced data. Hyperparameter tuning did not improve performance; we concluded that the base model without hyperparameter tuning performed already quite well on our data.

For the implementation of our BERT model, we used different regularizations (Early Stopping, Weight Decay and Dropout) to help with model generalization due to limited training data. RoBERTa served as a strong general-purpose baseline, while FinBERT offered insights into the benefit of domain-specific pre-training. Both models were fine-tuned using the classification head on the risk severity dataset. The models were trained using the AdamW optimizer with a batch size of 8, a learning rate of 2e-5, and early stopping based on validation loss. Training was performed in Google Colab with GPU acceleration enabled.

We implemented KeyBERT using the 'ProsusAI/finbert' model to align key phrase extraction with financial domain terminology. This ensured that extracted phrases reflected semantically relevant financial language. For each severity class, we randomly sampled 5 documents from the test set. Keyphrases were extracted using a bi-gram range (ngram_range=(1, 2)) and the Maximal Marginal Relevance (MMR) setting with a diversity score of 0.7 to ensure phrase variety. The top 5 phrases per document were returned along with cosine similarity scores.

Intermediate results

This section outlines key observations and insights obtained during development process, highlighting what was discovered before final model selection and evaluation.

We analysed the distribution of text length across severity levels. In figure 1, 'HIGH' and 'MEDIUM' severity disclosures tend to be longer and exhibit greater variability in length compared to 'LOW' and 'NONE' categories. This supports the observation that more severe events are often described in greater detail, potentially providing richer linguistic signals for the model to learn from. Outliers in the 'HIGH' and 'MEDIUM' categories also suggest that some documents contain exceptionally extensive narrative disclosures.

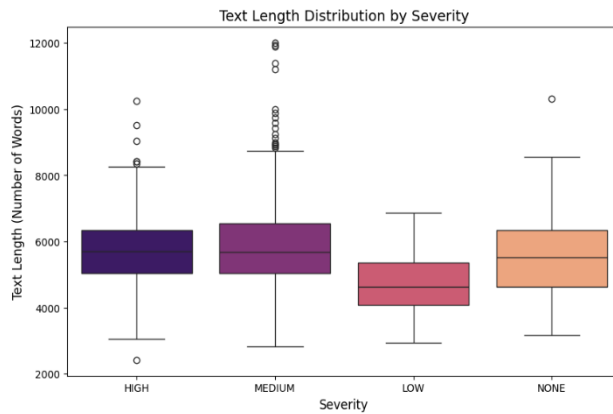


Figure 1: Box plot of text length by severity



Figure 2: Heatmap of Risk categories vs Risk severity

As part of exploratory analysis, we examined the relationship between risk categories and assigned severity levels. As shown in Figure 2, certain categories such as ‘OPERATIONAL’, ‘MARKET’, and ‘DEBT’ skewed heavily toward ‘MEDIUM’ or ‘HIGH’ severity, while others like ‘TAX’ and ‘LEGAL’ had a more even or lower-severity distribution. Although this variable was not included in the modelling pipeline, the heatmap helped contextualize severity trends across financial disclosure types.

Furthermore, after applying the relevant preprocessing steps (text normalisation, lemmatization and stop words removal), we also analysed the top frequent terms within each severity level. As shown in Table 3, terms like “risk,” “financial,” and “business” appear frequently in ‘HIGH’ and ‘MEDIUM’ severity disclosures, suggesting their importance in describing substantive financial threats. Conversely, ‘LOW’ and ‘NONE’ categories were dominated by more routine terms such as “statement,” “plan,” and “control.” This analysis further validated our decision to retain domain-specific vocabulary during preprocessing and guided our stop word filtering strategy.

Table 3: Top terms after preprocessing

	HIGH	MEDIUM	LOW	NONE
1	Risk	Risk	Dollar	Financial
2	Financial	Financial	Tax	Statement
3	Business	Business	Financial	Control
4	Result	Operation	Million	Reporting
5	operation	dollar	cash	Plan

Several features were engineered based on observations from EDA. These included text length, token count, and the number of financial-specific terms per disclosure. For instance, ‘HIGH’ severity documents were generally longer and contained a higher density of domain-relevant phrases, as seen in earlier term frequency and bigram analyses. However, these features were not included in model training, as the project focused on a purely text-based NLP pipeline. This choice ensured that the models learned directly from language patterns rather than relying on potentially unavailable or engineered metadata.

Cross-validation strategy

We initially explored a simple train/validation strategy due to hardware limitations, as we used transformer models. We believed that train/validation strategy would be computationally less expensive than a K-fold cross validation strategy for our transformer models. However, our dataset being relatively small and imbalanced, this led to poor performance as the models had even less data to train on. Hence, we switched to K-fold (with 5 fold) and applied stratification when the data is split for each fold.

Baseline models

Before applying data augmentation to reduce dataset imbalances, we tested TF-IDF and BoW with Logistic regression (LR) and SVM. Table 4 shows the results before and after. We saw improvement in both accuracy and F1-score when data augmentation was applied.

Table 4: Performance Results (Accuracy) Before and After Data Augmentation

	BoW LR	TF-IDF LR	BoW SVM	TF-IDF SVM
Before Data augmentation	0.60	0.57	0.61	0.63
After Data augmentation	0.71	0.69	0.72	0.75

We also experimented with less features (we set the maximum feature parameter of text vectorizers (BoW and TF-IDF) to 3000 instead on 5000). This however led to poor performance where accuracy ranged from 0.50 to 0.60 compared to what we have seen in Table 4.

Transformer (BERT) models

For our transformer models, FinBERT and RoBERTa, we initially explored the models 5 epochs without any regularization. For both FinBERT and RoBERTa, we observed that the model was still improving, and we decided to increase the number of epochs to 7. Additionally, we saw fluctuations in the losses; in some cases, validation would increase while training loss would be decreasing after a few epochs (usually 3 or 4). We, hence decided to introduce early stopping. We also experimented with Weight Decay and Dropout as we wanted the model to generalise well and not overfit the training data. This led to improvements on validation performance. We finally added class weights due to the imbalanced data.

We suspected overfitting during model implementation, but we believe it was more of a case of the models learning the majority classes too well. This led to the model defaulting to the majority classes when it was not sure about a prediction.

Keyword/key phrases extraction (YAKE vs KeyBERT)

As part of the project, we aimed to identify key risk factors within financial disclosures. We implemented SHAP and LIME to interpret model predictions and highlight which terms most influenced classification outcomes. These explanations helped validate whether the model's focus aligned with risk-relevant content, and we also planned to use them as tools to identify the key factors for particular severity level. In parallel, we explored unsupervised keyword extraction using YAKE and KeyBERT. We ran experimental tests to decide whether to choose YAKE or KeyBERT and ultimately opted for KeyBERT as we observed more finance-related results. The table below illustrates a sample output comparison.

YAKE relies on statistical features such as term position and frequency, KeyBERT uses contextual embeddings for more semantically informed phrase extraction. KeyBERT showed much better performance especially when backed by FinBERT.

Table 5: KeyBERT vs YAKE

KeyBERT	YAKE
Exacerbated	Company
Pandemic	Company financial
Inflationary	Company Ability
Profitability	Financial condition
Results	Financial

All three (SHAP, LIME and KeyBERT) has limitations when acting as a risk identification tool. Further experiments should be conducted or we should identify other tools.